■ CINEGO ARCHITECTURE & VIVA DEFENSE GUIDE

End-to-End Data Flow: Where Data is Stored, Image Handling, and API Access

Complete Viva Answers and Presentation Script for Evaluators

1. Question 1: "When a user books a ticket in UI, WHERE is it stored?"

Direct Answer to Mam: *"Ma'am, booking a ticket in our React UI triggers a coordinated 4-table persistence pipeline inside our Oracle 21c Database, governed by automated database triggers and sequences."*

Here is the exact step-by-step lifecycle of where and how the booking is stored:

Step & Component	Target Oracle Table	What Gets Saved & Database Action
1. Booking Master Record (React UI → API → SQL)	**BOOKINGS**	Creates master booking record with BOOKING_ID from BOOKING_SEQ.NEXTVAL. Stores CUSTOMER_ID, SHOW_ID, NUM_SEATS, TOTAL_AMOUNT, FINAL_AMOUNT, and initial STATUS = 'PENDING'.
2. Instant Seat Lock (Database Trigger fires)	**SHOW_SEATS**	Automated Trigger **TRG_BOOK_SEAT** fires upon insert on BOOKINGS. It updates STATUS = 'BOOKED' on the selected SHOW_SEAT_ID, preventing any other user from selecting those seats.
3. E-Ticket Generation (Trigger Action)	**TICKETS**	Trigger **TRG_BOOK_SEAT** simultaneously generates an official ticket record in TICKETS using TICKET_SEQ.NEXTVAL with STATUS = 'ACTIVE', linking BOOKING_ID and SHOW_SEAT_ID.
4. Payment Confirmation (Simulated / Gateway)	**PAYMENTS** → **BOOKINGS**	Stores payment receipt in PAYMENTS (PAYMENT_SEQ.NEXTVAL, amount, method, status). Trigger **TRG_PAYMENT_CONFIRM** flips BOOKINGS.STATUS = 'CONFIRMED'.
5. Concessions (Optional) (Food & Snacks Cart)	**FOOD_ORDERS**	If snacks are selected, saves line items into FOOD_ORDERS with FOOD_ID, quantity, and subtotal linked to the parent BOOKING_ID.

2. Question 2: "Where are the photos taken from and where are they saved?"

Direct Answer to Mam: *"Ma'am, in modern enterprise software, high-resolution multimedia files are NEVER stored as raw binary BLOBs inside relational database tables. Storing heavy images in relational tables degrades query speed, causes database bloat, and crashes connection pools."*

The 3 Pillars of Image Architecture in CineGo:

1. Database Storage (Metadata URLs): The Oracle database stores optimized **CDN URI strings** in VARCHAR2(500) columns: MOVIES.POSTER_URL, MOVIES.BACKDROP_URL, FOOD.IMAGE_URL, and CUSTOMERS.PROFILE_IMAGE. The database stores the pointer; the browser fetches the optimized image asset directly from the CDN.

2. Media Source & CDN Providers: Posters and cinema backdrops are hosted on ultra-fast Content Delivery Networks (Unsplash Cinema CDN, Cloudinary/Supabase CDN, and local public web assets in /public/posters/). They leverage automatic WebP compression, lazy loading, and hardware GPU acceleration.

3. Dynamic QR Code Tickets (Generated On-the-Fly): The E-Ticket QR code shown at the cinema turnstile gate is **dynamically generated in real-time** by the qrcode.react library in the frontend and qrcode on the backend. It encodes the unique cryptographic hash: { bookingId: 5001, seats: ['A1','A2'], verifyToken: 'SECURE_HASH' }. No static QR images are stored, saving disk space!

3. Question 3: "How does the API access the Database and Frontend?"

Direct Answer to Mam: *"Ma'am, our system uses an Enterprise 3-Tier Client-Server Architecture: Frontend (Presentation Tier) ↔ REST API (Application Tier) ↔ Oracle Database (Data Tier)."*

Tier	Technology	How It Communicates & Operates
Tier 1: Presentation (Frontend)	React 18 + Vite + TypeScript	Uses **Axios HTTP Client** (apiClient.ts) configured with baseURL: '/api'. Sends asynchronous JSON payloads with JWT tokens: POST /api/bookings, POST /api/payments.
Tier 2: Application (REST API)	Node.js + Express + TypeScript	Listens on PORT 3000 / 10000. Intercepts incoming requests, validates schema using **Zod**, checks JWT security, and dispatches to business services (bookingService.ts).
Tier 3: Data Access (Database Connector)	Oracle Database 21c XE + oracledb Pool	The backend maintains an active **Oracle Connection Pool** (config/database.ts) via localhost:1522/XEPDB1. It invokes PL/SQL stored procedures (CREATE_BOOKING, PROCESS_PAYMENT) using safe **bind parameters** (:p_customer_id) to guarantee ACID atomicity and prevent SQL injection.

4. Word-for-Word Script to Explain to Mam (Viva Presentation)

When Mam asks you questions about where data is stored, how photos work, or how the API connects, speak this exact response:

"Respected Ma'am, let me walk you through the complete end-to-end flow of our CineGo application:

1. When a user books a ticket in our UI:
*The customer selects seats on our interactive Seat Map and clicks 'Pay'. The React frontend sends an HTTP POST request to our Express REST API at /api/bookings. The API calls our Oracle stored procedure CREATE_BOOKING using bind parameters. Inside Oracle 21c, the master record is inserted into the **BOOKINGS** table with a generated sequence ID. Immediately, an automated trigger **TRG_BOOK_SEAT** fires to update the **SHOW_SEATS** table to 'BOOKED' and generates an official admission entry in the **TICKETS** table. Upon payment completion, the **PAYMENTS** table records the receipt, and trigger **TRG_PAYMENT_CONFIRM** marks the booking 'CONFIRMED'.*

2. Regarding photo and media storage:
*We follow industry standards by storing **image URI pointers** in Oracle's VARCHAR2(500) columns—specifically POSTER_URL and BACKDROP_URL in the MOVIES table. The actual binary images are served through fast CDNs with modern WebP compression. This keeps our database lightweight and high-performing. Furthermore, our E-Ticket QR codes are dynamically generated in real-time by the frontend using booking encryption hashes, so no static QR images take up server disk space.*

3. How the API accesses the database and frontend:
*Our frontend connects to the backend over REST HTTP endpoints via Axios. The Node.js backend maintains an active **Oracle connection pool** via the official oracledb native driver. Every database action is executed through compiled PL/SQL stored procedures and explicit cursors with ACID transaction commits. Once Oracle commits the transaction, the API returns a JSON response to React, which dynamically routes the user to their confirmed E-Ticket with confetti celebration."*

5. Rapid-Fire Teacher Questions & 10-Second Answers

Mam: "Where is the customer password saved?"

You: "Ma'am, in the CUSTOMERS table under PASSWORD_HASH VARCHAR2(255). We never store plain text passwords; they are hashed using secure cryptographic hashing."

Mam: "What sequence generates the Booking ID?"

You: "Ma'am, BOOKING_SEQ, which starts at 5000 so customer-facing booking numbers are clean and professional."

Mam: "What happens if the user closes the tab before paying?"

You: "Ma'am, the seat remains temporarily locked with a 10-minute timestamp (LOCKED_AT in SHOW_SEATS). If payment is not completed, our scheduled unlock function automatically frees the seat back to 'AVAILABLE'."

Mam: "Can you show me the SQL tables right now?"

You: "Yes Ma'am! We can open SQL*Plus or SQL Developer and run SELECT * FROM BOOKINGS; and SELECT * FROM SHOW_SEATS WHERE STATUS='BOOKED'; to see the live records!"

CineGo Movie Ticket Booking System • Oracle Database 21c XE • React TypeScript • Express REST API • Author: Ruthra K